

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Muhammad Syaiful Kalam - 5024241093

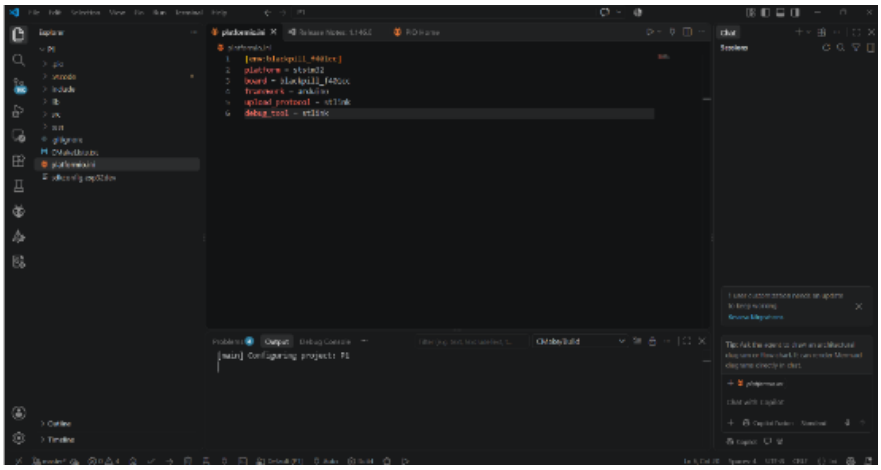
2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Langkah Kerja:

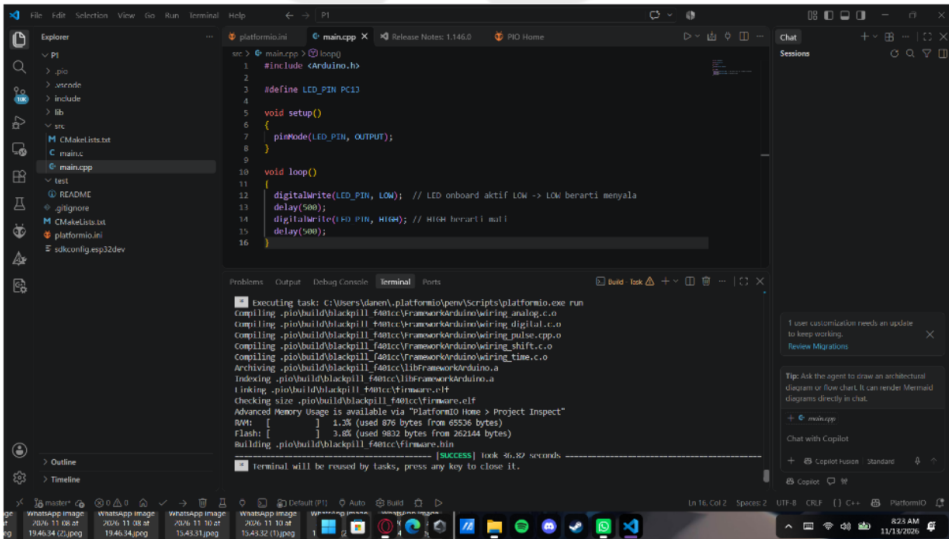
1. Melakukan config `platformio.ini` dengan menggunakan kode berikut:



Gambar 1: Konfigurasi PlatformIO untuk STM32 Blackpill

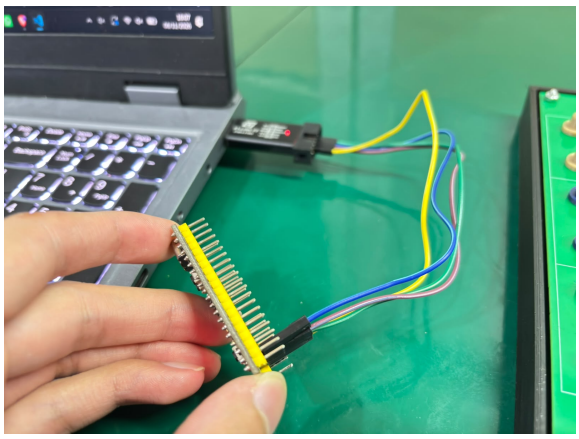
2. Setelah itu menhubungkan pin SWDIO, SWCLK, GND, dan 3V3 ST-Link ke Blackpill, lalu colokkan ST-Link ke komputer melalui USB

3. Setelah itu masukan kode program Blink pada file `src/main.cpp`, setelah itu Build dan Upload melalui PlatformIO.:



Gambar 2: Kode Blink STM32 pada `main.cpp` dan hasil Build di PlatformIO

4. Setelah itu amati LED onboard pada pin PC13 ditandai dengan nyala selama 500ms, mati selama 500ms, berulang terus-menerus.



Gambar 3: Sambungan Blackpill dengan ST-Link

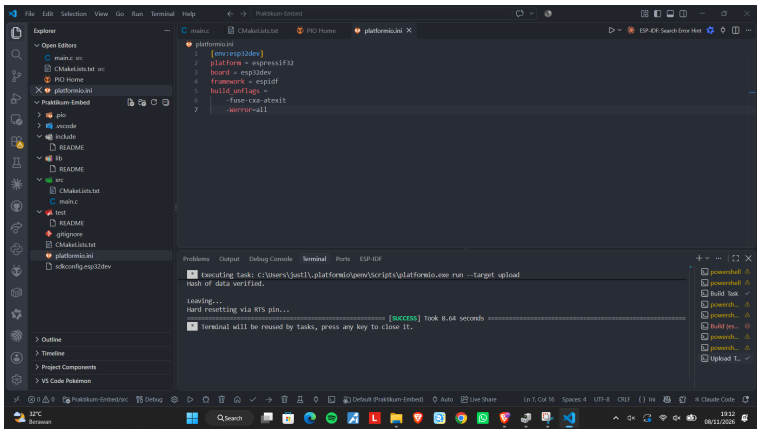


Gambar 4: Blackpill saat terhubung ke ST-Link

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

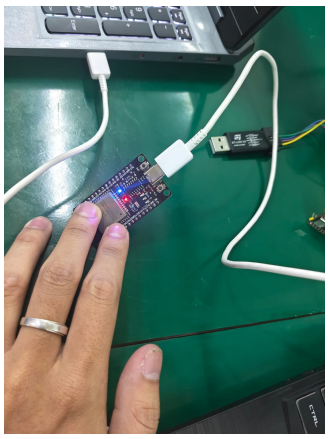
Langkah Kerja:

1. Mengganti isi `platformio.ini` dengan konfigurasi ESP32 dan framework ESP-IDF, Setelah itu tunggu proses refresh-index konfigurasi PlatformIO selesai :



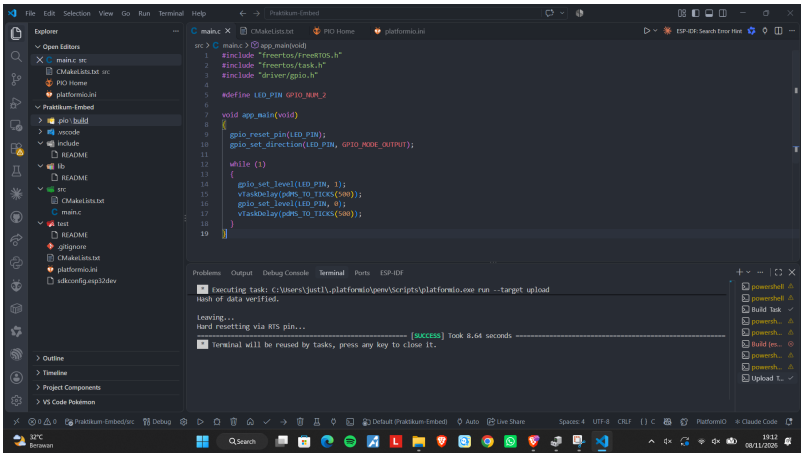
Gambar 5: Konfigurasi ESP-IDF dan keluaran proses upload di PlatformIO

2. Mencolokkan ESP32 DevKit ke komputer lewat USB dan memastikan port terdeteksi.



Gambar 6: ESP32 yang tersambung ke komputer melalui USB

3. Menghapus file `src/main.cpp` dan membuat file baru `src/main.c`, kemudian menulis kode program Blink ESP-IDF, Setelah itu melakukan Build dan Upload:



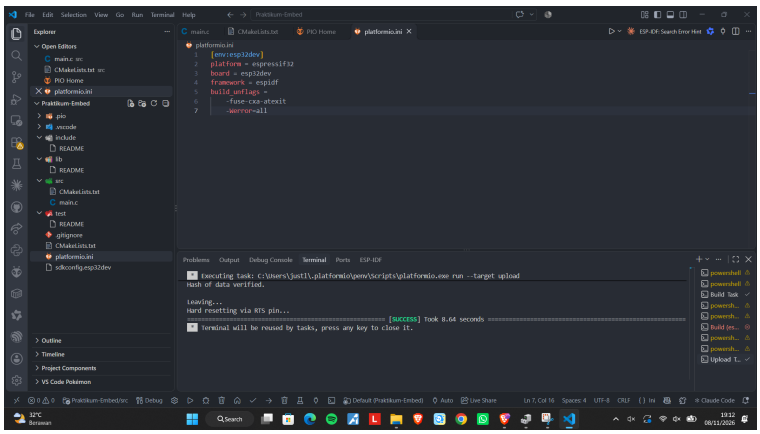
Gambar 7: Program Blink ESP32 dengan ESP-IDF selesai diunggah dengan status `[SUCCESS]`

4. Mengamati LED onboard ESP32 (GPIO 2) berkedip dengan interval 500ms.

1.3 Percobaan 3: Pull-up dan Pull-down – Eksternal vs Internal (ESP32 + Framework Arduino)

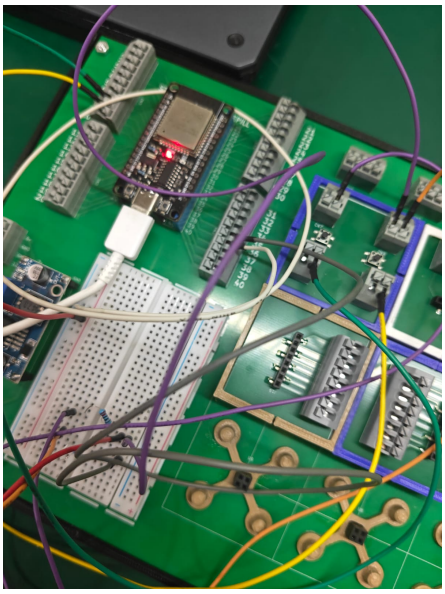
Langkah Kerja:

1. Mengganti isi `platformio.ini` ke konfigurasi Arduino setelah itu menghapus `src/main.c` dan membuat kembali file `src/main.cpp`.:



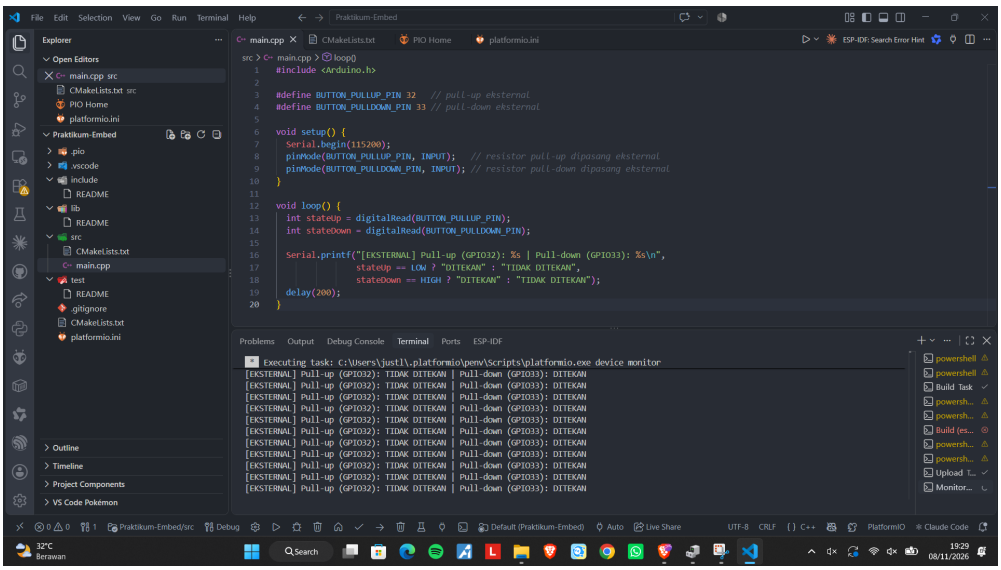
Gambar 8: Konfigurasi di PlatformIO

2. **Bagian A – Pull-up/Pull-down Eksternal:** Memasang dua pushbutton di GPIO 32 (pull-up eksternal dengan resistor 10kΩ ke 3.3V) dan GPIO 33 (pull-down eksternal dengan resistor 10kΩ ke GND).



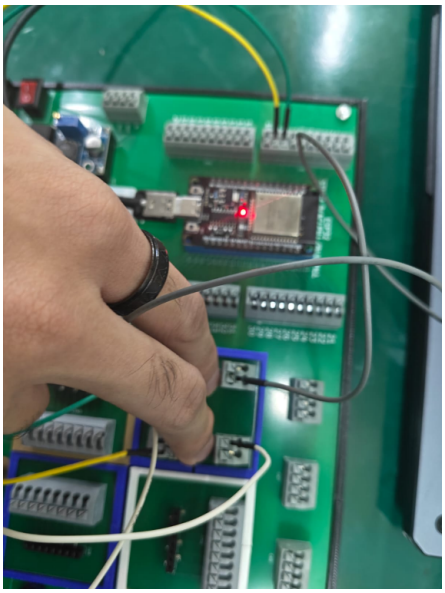
Gambar 9: Sambungan ESP32 ke area tombol pada board trainer

3. Menulis Kode Program A (Eksternal) pada `src/main.cpp` setelah itu melakukan Build dan Upload, kemudian menekan kedua tombol dan mencatat hasil dari Serial Monitor:



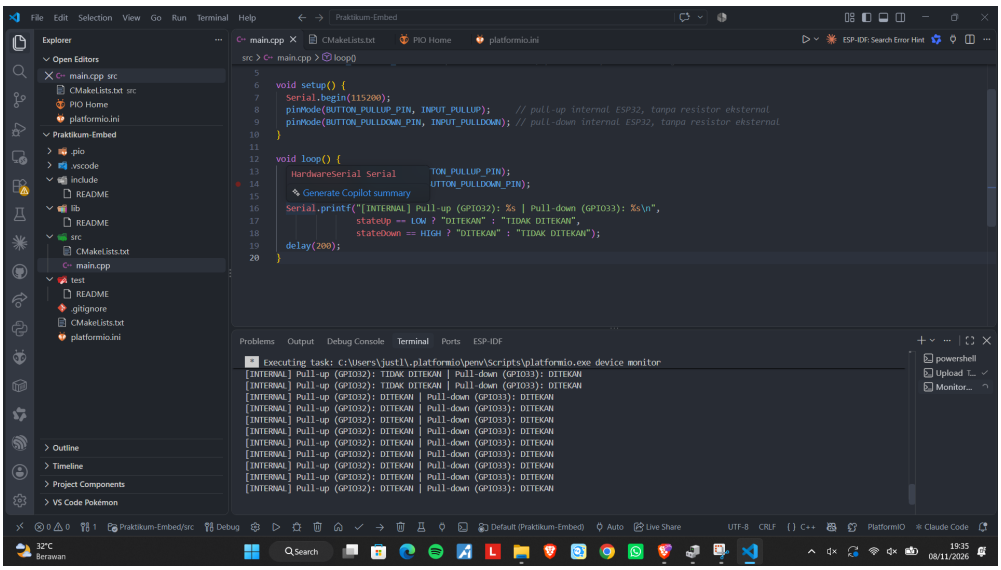
Gambar 10: Kode pull-up dan pull-down eksternal pada `main.cpp`

4. **Bagian B – Pull-up/Pull-down Internal:** Melepas kedua resistor eksternal dan mengaktifkan resistor internal melalui konfigurasi GPIO. Tombol pull-up tetap menghubungkan GPIO 32 ke GND saat ditekan, sedangkan tombol pull-down tetap menghubungkan GPIO 33 ke 3.3V saat ditekan. Koneksi ini sesuai dengan penggunaan `INPUT_PULLUP` dan `INPUT_PULLDOWN` pada kode.

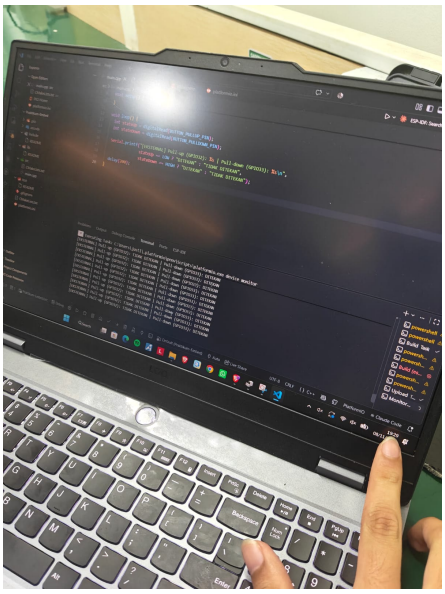


Gambar 11: Pengujian tombol pada board trainer

5. Mengganti isi `src/main.cpp` dengan Kode Program B (Internal) setelah itu melakukan Build dan Upload, kemudian menekan kedua tombol dan mencatat hasil dari Serial Monitor.:



Gambar 12: Kode pull-up dan pull-down internal pada `main.cpp`



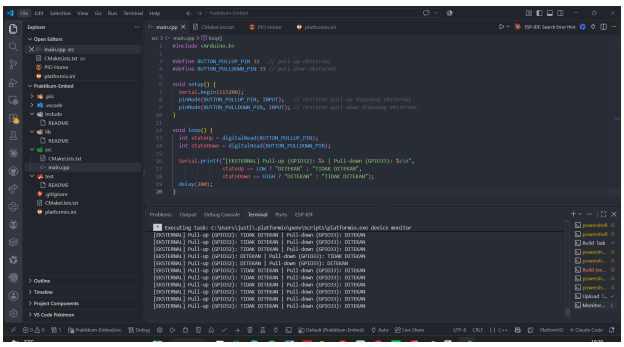
Gambar 13: Dokumentasi pengujian tombol dan pembacaan Serial Monitor

6. Membandingkan hasil kedua bagian. Pada dokumentasi pengujian internal, GPIO 33 masih terbaca “DITEKAN”, sehingga hasil pull-down internal belum sama dengan keadaan yang diharapkan.

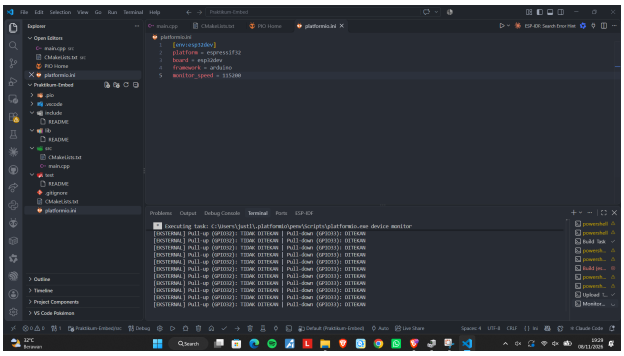
1.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Langkah Kerja:

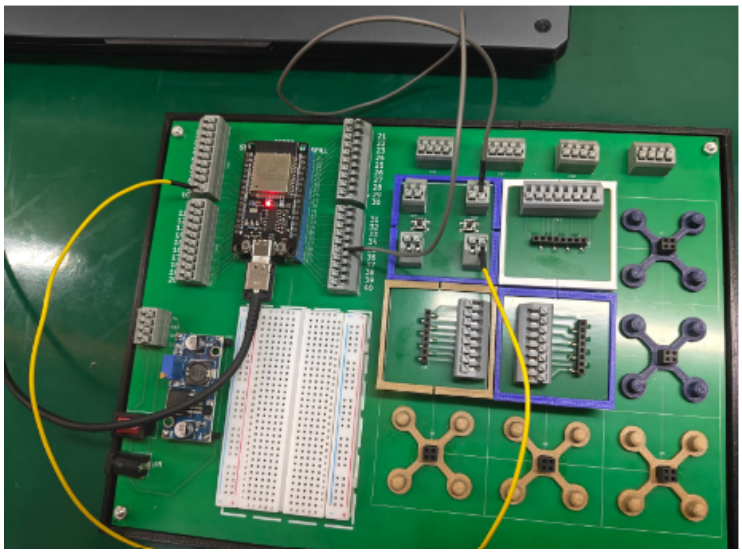
1. Memasang satu pushbutton pada GPIO 25 ESP32 dengan konfigurasi pull-up internal (satu kaki ke GPIO, kaki lain ke GND).



Gambar 14: Pembacaan tombol dengan resistor eksternal



Gambar 15: Pembacaan tombol dengan resistor internal

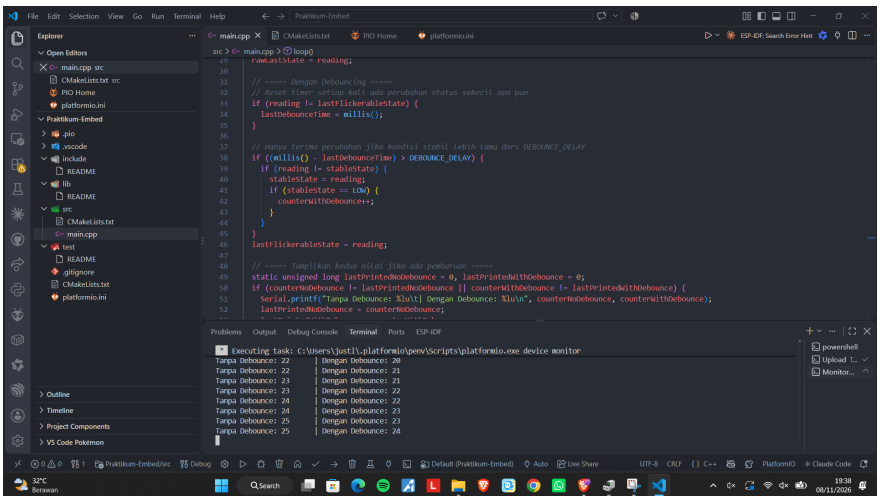


Gambar 16: Susunan ESP32 dan board trainer untuk pengujian tombol

2. Konfigurasi `platformio.ini` tetap sama seperti Percobaan 3 (ESP32 Arduino), sehingga tidak perlu diubah.

3. Menulis kode program perbandingan counter tanpa debouncing dan dengan debouncing pada `src/main.cpp` setelah itu melakukan Build dan Upload, kemudian menekan tombol satu kali dan membandingkan nilai kedua counter di Serial Monitor.:

4. Mengamati kedua counter pada Serial Monitor sambil menekan tombol. Pada tangkapan layar, nilai tanpa debounce bertambah hingga 25, sedangkan nilai dengan debounce mencapai 24.

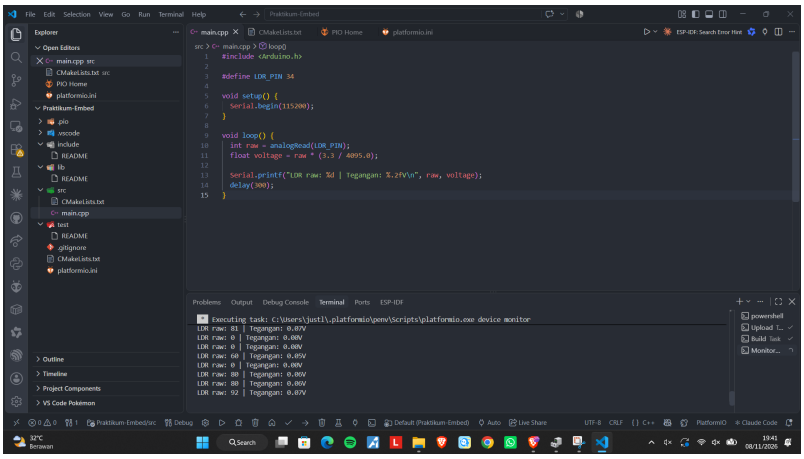


Gambar 17: Program debounce dan nilai kedua counter pada Serial Monitor

1.5 Percobaan 5: Akses GPIO Analog – Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

Langkah Kerja:

1. Memasang LDR bersama resistor 10kΩ dalam konfigurasi pembagi tegangan: satu kaki LDR ke 3.3V, kaki lain ke GPIO 34 dan ke GND melalui resistor 10kΩ.



Gambar 18: Pembacaan ADC LDR yang terekam pada Serial Monitor

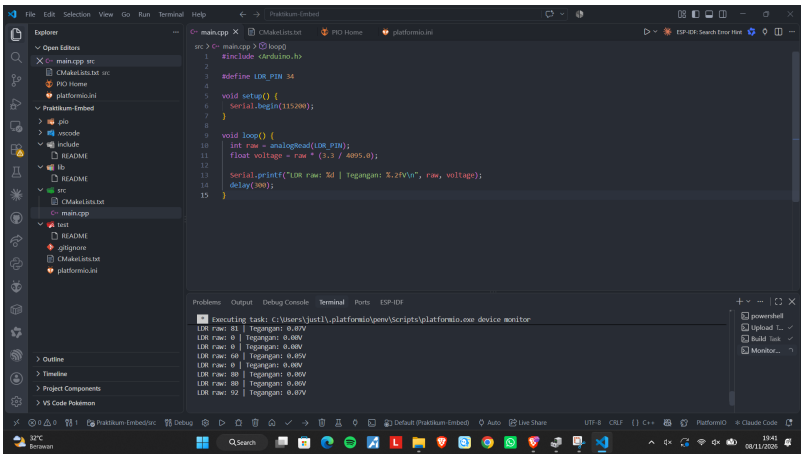
2. Konfigurasi `platformio.ini` tetap sama (ESP32 Arduino). Kecepatan Serial Monitor diatur ke 115200 baud melalui parameter `monitor_speed`.

3. Menulis kode program pembacaan ADC pada `src/main.cpp` setelah itu melakukan Build dan Upload, kemudian membuka Serial Monitor.:

Listing 1: Kode Program Pembacaan ADC dengan LDR

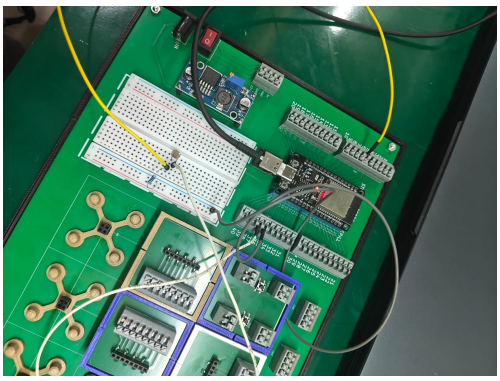
```
1
2 #include <Arduino.h>
3
4
5
6 #define LDR_PIN 34
```

```
7
8
9
10 void setup() {
11
12   Serial.begin(115200);
13
14 }
15
16
17
18 void loop() {
19
20   int raw = analogRead(LDR_PIN);
21
22   float voltage = raw * (3.3 / 4095.0);
23
24
25
26   Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
27
28   delay(300);
29
30 }
31
32
```

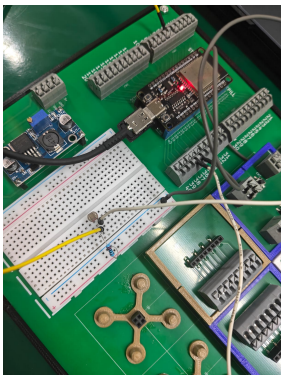


Gambar 19: Pembacaan ADC LDR yang terekam pada Serial Monitor

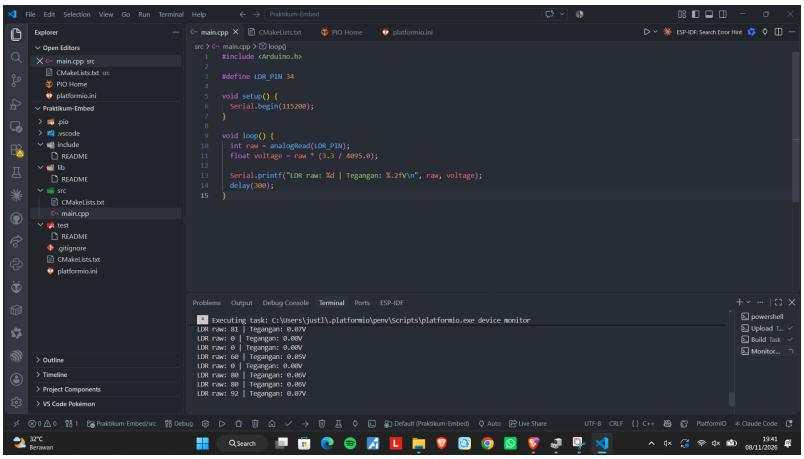
4. Mengamati nilai ADC dan tegangan yang dicetak pada Serial Monitor. Pada tangkapan layar, nilai raw yang terlihat berkisar 0–92 dan tegangan hasil perhitungan berkisar 0,00–0,07 V.



Gambar 20: Susunan ESP32 dan breadboard untuk pembacaan LDR



Gambar 21: Detail LDR dan resistor pada breadboard



Gambar 22: Pembacaan ADC LDR yang terekam pada Serial Monitor

2 Analisis Hasil Percobaan

Pada Percobaan 1 dan 2, LED onboard pada STM32 Blackpill dan ESP32 berhasil berkedip dengan interval nyala dan mati masing-masing 500 ms. Perbedaan keduanya terletak pada framework yang digunakan. STM32 menggunakan pendekatan STM32duino, sedangkan ESP32 menggunakan ESP-IDF dengan fungsi `app_main()` dan `vTaskDelay()` dari FreeRTOS. Selama proses pengujian, sempat terjadi kendala saat proses *build* yang disebabkan oleh konfigurasi lingkungan PlatformIO. Kendala tersebut berhasil diatasi melalui pemasangan ulang framework dan pembuatan ulang proyek.

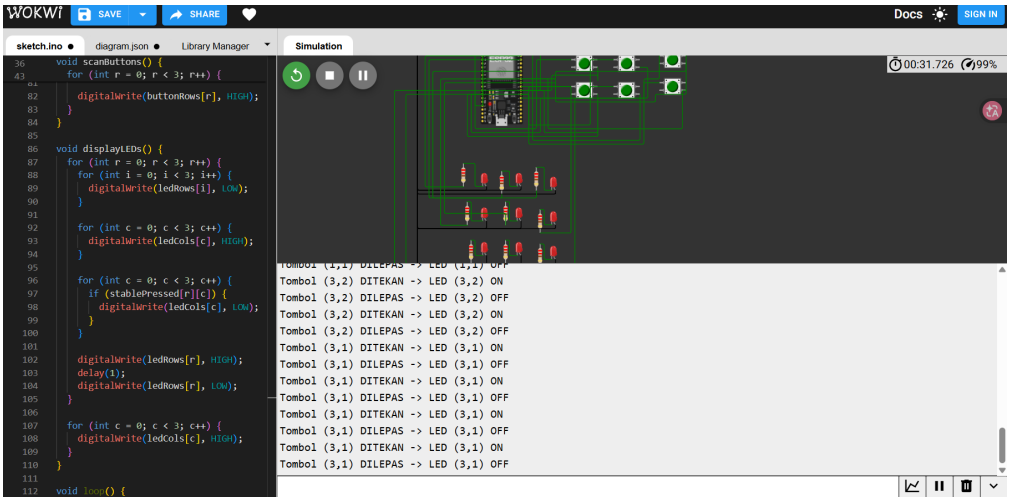
Pada Percobaan 3, pengujian pull-up dan pull-down dilakukan menggunakan resistor eksternal maupun internal. Secara teori, pull-up menghasilkan kondisi HIGH saat tombol dilepas dan LOW saat ditekan, sedangkan pull-down memiliki kondisi sebaliknya. Hasil pengujian eksternal menunjukkan pembacaan sesuai teori. Namun, pada pengujian internal, GPIO 33 masih terbaca dalam kondisi DITEKAN pada dokumentasi pengamatan. Hal ini menunjukkan bahwa konfigurasi internal belum sepenuhnya menghasilkan pembacaan yang diharapkan. Penggunaan resistor pull tetap diperlukan agar pin memiliki kondisi logika yang stabil dan tidak mengalami *floating*.

Pada Percobaan 4, pengujian debouncing menunjukkan bahwa pembacaan tombol tanpa penyaringan dapat menghasilkan lebih dari satu hitungan dalam satu kali penekanan. Hal ini disebabkan oleh pantulan kontak mekanis pada tombol (*bouncing*). Berdasarkan hasil pengamatan, counter tanpa debounce mencapai 25, sedangkan counter dengan debounce mencapai 24. Perbedaan tersebut menunjukkan bahwa penerapan debouncing dapat membantu mengurangi pembacaan ganda akibat perubahan sinyal yang tidak stabil.

Pada Percobaan 5, sensor LDR berhasil dibaca menggunakan ADC pada GPIO 34 ESP32. Nilai ADC yang terlihat pada Serial Monitor berada pada rentang 0–92, dengan tegangan sekitar 0,00–0,07 V. Pembacaan tersebut menunjukkan bahwa ESP32 dapat mengubah tegangan analog dari rangkaian LDR menjadi nilai digital melalui ADC 12-bit dengan rentang 0–4095. Perubahan intensitas cahaya akan memengaruhi resistansi LDR dan tegangan yang diterima oleh pin ADC.

3 Hasil Tugas Modul

3.1 Perakitan Rangkaian pada Board Trainer



Gambar 23: Rangkaian matrix pushbutton dan LED 3x3 pada board trainer

3.2 Implementasi Program Matrix Pushbutton dan LED

Listing 2: Kode Program Matrix Pushbutton dan LED 3x3

```
const int buttonRows[3] = {13, 14, 16};
const int buttonCols[3] = {17, 18, 19};

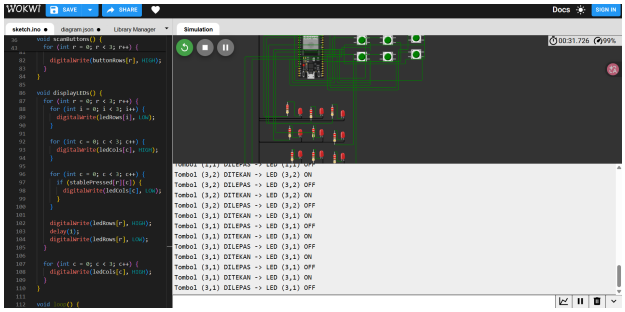
const int ledRows[3] = {21, 22, 23};
```

```
5  const int ledCols[3] = {25, 26, 27};
6
7  const unsigned long debounceDelay = 50;
8
9  bool rawPressed[3][3] = {};
10 bool stablePressed[3][3] = {};
11 unsigned long lastDebounceTime[3][3] = {};
12
13 void setup() {
14   Serial.begin(115200);
15
16   for (int r = 0; r < 3; r++) {
17     pinMode(buttonRows[r], OUTPUT);
18     digitalWrite(buttonRows[r], HIGH);
19   }
20
21   for (int c = 0; c < 3; c++) {
22     pinMode(buttonCols[c], INPUT_PULLUP);
23   }
24
25   for (int r = 0; r < 3; r++) {
26     pinMode(ledRows[r], OUTPUT);
27     digitalWrite(ledRows[r], LOW);
28   }
29
30   for (int c = 0; c < 3; c++) {
31     pinMode(ledCols[c], OUTPUT);
32     digitalWrite(ledCols[c], HIGH);
33   }
34 }
35
36 void scanButtons() {
37   unsigned long now = millis();
38
39   for (int r = 0; r < 3; r++) {
40     digitalWrite(buttonRows[r], HIGH);
41   }
42
43   for (int r = 0; r < 3; r++) {
44     digitalWrite(buttonRows[r], LOW);
45     delayMicroseconds(50);
46
47     for (int c = 0; c < 3; c++) {
48       bool reading = (digitalRead(buttonCols[c]) == LOW);
49
50       if (reading != rawPressed[r][c]) {
51         rawPressed[r][c] = reading;
52         lastDebounceTime[r][c] = now;
53       }
54
55       if (now - lastDebounceTime[r][c] >= debounceDelay) {
56         if (stablePressed[r][c] != rawPressed[r][c]) {
57           stablePressed[r][c] = rawPressed[r][c];
58
```

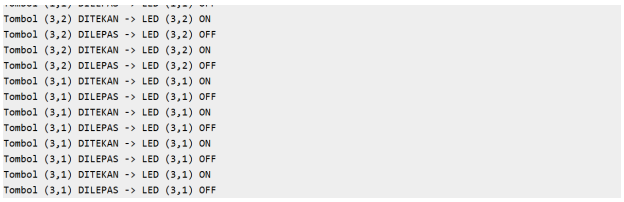
```
59            Serial.print("Tombol (");
60            Serial.print(r + 1);
61            Serial.print(",");
62            Serial.print(c + 1);
63            Serial.print(") ");
64
65            if (stablePressed[r][c]) {
66              Serial.print("DITEKAN -> LED (");
67              Serial.print(r + 1);
68              Serial.print(",");
69              Serial.print(c + 1);
70              Serial.println(") ON");
71            } else {
72              Serial.print("DILEPAS -> LED (");
73              Serial.print(r + 1);
74              Serial.print(",");
75              Serial.print(c + 1);
76              Serial.println(") OFF");
77            }
78          }
79        }
80      }
81
82      digitalWrite(buttonRows[r], HIGH);
83    }
84  }
85
86  void displayLEDs() {
87    for (int r = 0; r < 3; r++) {
88      for (int i = 0; i < 3; i++) {
89        digitalWrite(ledRows[i], LOW);
90      }
91
92      for (int c = 0; c < 3; c++) {
93        digitalWrite(ledCols[c], HIGH);
94      }
95
96      for (int c = 0; c < 3; c++) {
97        if (stablePressed[r][c]) {
98          digitalWrite(ledCols[c], LOW);
99        }
100     }
101
102     digitalWrite(ledRows[r], HIGH);
103     delay(1);
104     digitalWrite(ledRows[r], LOW);
105   }
106
107   for (int c = 0; c < 3; c++) {
108     digitalWrite(ledCols[c], HIGH);
109   }
110 }
111
112 void loop() {
```

```
113   scanButtons();
114   displayLEDs();
115 }
```

3.2.1 Dokumentasi Hasil Tugas Modul



Gambar 24: Rangkaian LED



Gambar 25: Keluaran pembacaan tombol pada Serial Monitor

3.3 Jawaban Pertanyaan Analisis

1. **Mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton?**

   Debouncing diperlukan karena tombol mekanis dapat menghasilkan perubahan sinyal yang tidak stabil ketika ditekan maupun dilepaskan. Kondisi ini disebut *bouncing*, yaitu pantulan kontak yang dapat menyebabkan satu kali penekanan terbaca sebagai beberapa input. Dengan menerapkan debouncing selama 50 ms, mikrokontroler hanya memproses perubahan tombol setelah sinyal berada dalam kondisi stabil sehingga pembacaan menjadi lebih akurat.

2. **Bagaimana konfigurasi pull-up/pull-down memengaruhi pembacaan tombol matrix?**

   Konfigurasi pull-up membuat pin input berada pada kondisi HIGH ketika tombol tidak ditekan dan berubah menjadi LOW ketika tombol ditekan pada baris yang aktif LOW. Konfigurasi ini digunakan agar pin memiliki acuan tegangan yang jelas dan tidak mengalami kondisi *floating*. Dengan demikian, pembacaan tombol menjadi lebih stabil dan perubahan status LED dapat mengikuti kondisi tombol dengan benar.

3. **Apa perbedaan digital input dan analog input (ADC)?**

   Digital input hanya membaca dua kondisi logika, yaitu HIGH dan LOW, sehingga cocok digunakan untuk mendeteksi penekanan tombol. Sementara itu, analog input menggunakan ADC untuk mengubah tegangan yang bervariasi menjadi nilai digital. Pada ESP32, ADC memiliki resolusi 12-bit dengan rentang nilai 0–4095. Contohnya pada sensor LDR, perubahan intensitas cahaya dapat dibaca melalui perubahan tegangan sehingga menghasilkan nilai ADC yang berbeda.

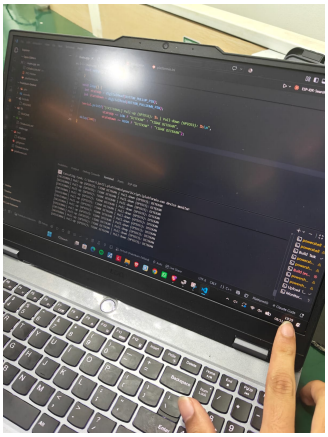
4 Kesimpulan

Berdasarkan keseluruhan rangkaian percobaan yang telah dilakukan, dapat disimpulkan bahwa pengendalian *output* LED dasar pada STM32 Blackpill (STM32duino) dan ESP32 (ESP-IDF) menghasilkan pola yang sama dengan perbedaan struktur program, di mana ESP-IDF menggunakan *task*

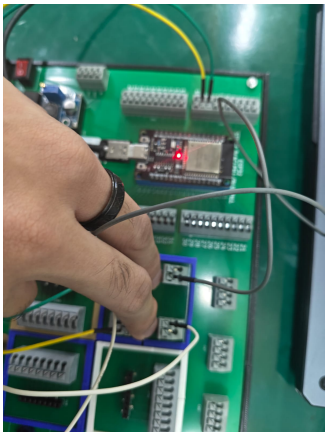
FreeRTOS yang tidak memblokir proses lain. Pada pembacaan tombol, penerapan resistor *pull-up* atau *pull-down* terbukti wajib untuk mencegah kondisi *floating*, sementara teknik *debouncing* dengan jeda optimal 50 ms sangat efektif meredam getaran kontak mekanis agar tidak terjadi pembacaan ganda. Selain itu, berbeda dari *input* digital yang hanya mengenali logika `HIGH` dan `LOW`, fitur ADC 12-bit pada ESP32 mampu mengonversi tegangan kontinu sensor LDR menjadi 4096 tingkatan nilai untuk merepresentasikan gradasi intensitas cahaya secara proporsional. Secara keseluruhan, praktikum dinyatakan berhasil dan sesuai teori, karena satu-satunya kendala yang sempat muncul murni berasal dari konfigurasi lingkungan pengembangan saat tahap *build*.

5 Lampiran

5.1 Dokumentasi Saat Praktikum



Gambar 26: Dokumentasi proses pemrograman dan upload



Gambar 27: Pengujian tombol pada board trainer saat praktikum